

BOOMI BOSS ENGINE

MASTER REFERENCE & ARCHITECTURE MANUAL

User, Architecture, Testing, Keyboard Shortcuts & Future Roadmap

Purpose

A single, readable reference manual for the current workspace, architecture, master shortcuts, testing workflow, and future roadmap. This document preserves the provided project content and restructures diagrams into Word-friendly visual blocks.

Contents

1. Project Folder & Workspace Directory Map
2. Core System Architecture & Data Flow
3. Master Shortcuts Reference
4. How to Run, Test, and Package the App
5. Future Roadmap & Unimplemented Extensions

1. Project Folder & Workspace Directory Map

Your workspace is organized as a clean, modular, production-grade Electron application:

Path / File	Purpose
knowledge/	Local intelligence files (zero-latency RAG).
knowledge/scenarios.json	The Master Scenario Bank containing 300+ exact-match technical interview questions and expert answers for sub-10ms local retrieval.
knowledge/resume.md	The candidate's real professional history, acting as the first-person truth persona layer.
knowledge/job-description.md	Target role parameters and technical focus areas used to shape relevance.
test/	Automated test harnesses.
test/engine.test.js	Core engine unit tests covering state machine transitions, rolling conversation memory, confidence scoring, and local scenario bypasses.
test/diagnostics.test.js	Validation tests for the JSONL/TXT interaction timeline logger.
test/audio-pipeline.test.js	Audio constraints validation and Deepgram streaming URL builder checks.
logs/	Timestamped session logs capturing interaction timelines when debug diagnostics are active.
dist/	Output directory generated when packaging the application into a standalone Windows .exe via Electron Builder.
main.js	Backend Electron process managing window geometry, transparency/background colors, stealth screen-protection (setContentProtection), and global OS shortcut registrations.
index.html	Floating overlay frontend UI, dark-mode teleprompter styling, and IPC event listener orchestration.
engine.js	Pure logic Interview Engine (DOM-free), handling silence/speech boundaries, rolling conversation memory (last 4 turns), Groq streaming orchestration, and candidate response grading.
audio-pipeline.js	WebRTC audio configuration (noiseSuppression, echoCancellation, autoGainControl), Deepgram WebSocket integration (filler_words=false), and candidate personal microphone capture (startCandidateAudio).
diagnostics.js	Observability module tracking event sequences and performance metrics.
domain-vocabulary.js	Single source of truth for domain-specific STT

Path / File	Purpose
	keyterms and linguistic incomplete hooks.
package.json	Project dependencies (@deepgram/sdk, electron) and build scripts (npm run build).

2. Core System Architecture & Data Flow

The application relies on a parallelized, high-speed architecture designed to eliminate cloud latency and present a seamless teleprompter experience:



Key Architectural Pillars:

Local First, Cloud Second: Exact-match technical questions bypass network latency entirely using local keyword matching against scenarios.json.

Linguistic Locking: The engine inspects the final word of an utterance; if it ends in a preposition or connector (e.g., “between”, “into”), it unconditionally waits for the speaker to finish, preventing premature triggering.

Stealth God Mode: Window content protection (setContentProtection(true)) ensures the overlay remains invisible to screen-sharing and recording software like Zoom, Teams, Google Meet, or OBS.

Phase 10 Candidate Analysis: A secondary physical microphone pipeline (Alt+V) records the candidate's spoken answer and grades it against the expected answer via Groq (Alt+A), displaying an 8-second scorecard.

3. Master Shortcuts Reference

Hotkey	Action	Description
Alt + S	Toggle Listen	Pauses or resumes live audio transcription and engine processing.
Alt + C	Clear	Clears the current question buffer, transcript display, and answer box.
Alt + R	Regenerate	Forces a fresh AI generation for the current transcript.
Alt + M	Toggle Mode	Switches output style between Script (conversational prose) and Architect (concise bullet points).
Alt + V	Toggle Candidate Mic	Turns on/off your personal microphone analysis pipeline.
Alt + A	Analyze Candidate	Triggers the Groq Grader to evaluate your spoken answer against the expected answer, showing an 8-second scorecard.
Alt + P	Panic Mode	Instantly toggles window opacity between 0% and 100% as a stealth fail-safe.
Alt + Z	Click-Through	Toggles mouse click-through behavior so clicks pass through the overlay to underlying software.
Alt + X	Toggle Size	Expands or collapses the overlay window height.

4. How to Run, Test, and Package the App

Step 1 — Launch the Application

```
npm start
```

Step 2 — Run the Automated Test Suite

```
npm test
```

Step 3 — Build the Standalone Windows Installer (.exe)

```
npm run build
```

This generates your installer inside the dist/ folder using electron-builder.

5. Future Roadmap & Unimplemented Extensions

- 1. Vector Database Integration (RAG Expansion)** — Transitioning from static file-based Markdown and JSON stores to a lightweight local vector embedding database (such as SQLite-vss or Chroma) for semantic fuzzy searching across hundreds of complex project documents.
- 2. Multi-Domain Knowledge Packs** — Expanding beyond Boomi into dedicated modules for Cloud DevOps (AWS/Azure/Kubernetes), Full-Stack System Architecture, and Enterprise Microservices.
- 3. Advanced Candidate Metrics (Speech Analytics)** — Upgrading Phase 10 from simple accuracy scoring to real-time speech analysis tracking words-per-minute (WPM), filler-word frequency (“um”, “uh”), and vocal pacing.
- 4. Cloud Synchronization & Profile Switcher** — Adding a lightweight UI panel to dynamically switch candidate personas, target job descriptions, and scenario banks on the fly without restarting the application.

Reference Note

This manual was created from the complete content supplied by the user. Diagrams have been converted into Word-friendly visual flow blocks and tables so the document remains readable and editable.